

WAL CHAT / OPERAÇÃO

Manual interno de implementação e operação

Arquitetura isolada, implantação, contas de usuário,
Meta, Gemini, compliance, validação e rotina operacional.

AMBIENTE	HOMOLOGAÇÃO HTTPS
Versão	MVP 2026.07
Atualização	21 de julho de 2026

1. Objetivo e escopo

Este documento descreve como implantar, validar e operar o MVP do Wal Chat. A implantação de homologação usa uma instância isolada no servidor 64.181.178.125, sem compartilhar banco, Redis, rede Docker ou portas internas com os demais sistemas do servidor.

Endereços previstos:

- Aplicação: <https://wal-chat.64.181.178.125.nip.io>
- API Supabase: <https://api-wal-chat.64.181.178.125.nip.io>
- Webhook Meta: <https://wal-chat.64.181.178.125.nip.io/api/public/webhooks/instagram>

O MVP possui duas formas de uso:

1. **Demonstração:** apresenta todos os módulos com dados de exemplo e bloqueia envios reais.
2. **Operação real:** autenticação e banco reais; integrações Meta e Gemini são habilitadas somente após cadastrar credenciais válidas e concluir as etapas de aprovação externas.

2. Arquitetura isolada

Componente	Isolamento	Função
wal-chat-app-1	Container e porta local 4194	Interface e APIs TanStack Start
wal-chat-webhooks-1	Container próprio	Consumo dos eventos do Instagram
wal-chat-scheduler-1	Container próprio	Sequências, delays e campanhas
wal-chat-redis-1	Volume e rede exclusivos	Filas BullMQ
Supabase wal_chat_prod	Containers, banco e volumes exclusivos	Auth, Postgres, RLS, REST e Storage
Nginx	Virtual hosts exclusivos	HTTPS e proxy reverso

O Nginx é compartilhado apenas como porta de entrada do servidor. Dados e processos do Wal Chat permanecem isolados.

3. Estrutura no servidor

```
/opt/wal-chat/
├── app/                                # código e compose da aplicação
│   ├── .env.production                # secrets; permissão 600
│   └── docker-compose.production.yml
├── supabase-instance/
│   ├── status.env                    # chaves geradas; permissão 600
│   └── supabase/                     # config, migrations e seed
```

Nunca enviar .env.production, status.env, tokens Meta ou chaves Supabase para Git, e-mail ou chats.

4. Implantação inicial

4.1 Pré-requisitos

- Oracle Linux ARM64 com Docker e Docker Compose.
- Nginx e Certbot.
- Portas públicas 80 e 443.
- DNS nip.io resolvendo para o IP do servidor.

- Acesso SSH administrativo.

4.2 Preparar o Supabase exclusivo

No diretório `/opt/wal-chat/app`:

```
chmod +x deploy/*.sh
DEPLOY_ROOT=/opt/wal-chat ./deploy/prepare-supabase.sh
cd /opt/wal-chat/supabase-instance
npx --yes supabase@2.109.1 start
npx --yes supabase@2.109.1 status -o env > status.env
chmod 600 status.env
```

Na primeira inicialização, as migrations criam todas as tabelas, RLS, GRANTs, índices e o usuário de homologação.

4.3 Gerar o ambiente da aplicação

```
cd /opt/wal-chat/app
DEPLOY_ROOT=/opt/wal-chat ./deploy/render-production-env.sh
```

O script gera senhas e tokens aleatórios. O arquivo final deve permanecer com permissão `600`.

4.4 Subir a aplicação

```
cd /opt/wal-chat/app
docker compose --env-file .env.production \
  -f docker-compose.production.yml up -d --build
```

4.5 Publicar no Nginx e gerar HTTPS

```
sudo install -m 644 deploy/nginx/wal-chat.conf /etc/nginx/conf.d/wal-chat.conf
sudo nginx -t
sudo systemctl reload nginx
sudo certbot --nginx --non-interactive --agree-tos \
  --register-unsafely-without-email \
  -d wal-chat.64.181.178.125.nip.io \
  -d api-wal-chat.64.181.178.125.nip.io \
  --redirect
```

5. Contas e permissões

5.1 Papéis disponíveis

Papel	Uso indicado
<code>owner</code>	Titular do workspace e responsável pela conta Instagram
<code>admin</code>	Configura usuários, integrações, gatilhos e campanhas
<code>agent</code>	Opera Inbox, contatos e conteúdo
<code>viewer</code>	Consulta dashboard e insights sem operar automações

O RLS garante que cada usuário leia somente workspaces dos quais é membro.

5.2 Criar o primeiro titular

1. Acesse a aplicação e escolha **Criar agora**.
2. Informe nome, e-mail corporativo e senha forte.
3. O trigger `handle_new_user` cria automaticamente um workspace e adiciona o usuário como `owner`.

4. Valide que Dashboard e Configurações abrem normalmente.

Na homologação, a confirmação de e-mail está desativada. Antes do Live Mode, configure SMTP e ative confirmação de e-mail.

5.3 Adicionar um operador a um workspace

1. Crie o usuário pelo fluxo de cadastro ou pela API administrativa do Supabase.
2. Descubra o `id` do usuário em `auth.users`.
3. Obtenha o `id` do workspace.
4. Insira a associação:

```
insert into public.workspace_members (workspace_id, user_id, role)
values ('WORKSPACE_UUID', 'USER_UUID', 'agent')
on conflict (workspace_id, user_id)
do update set role = excluded.role;
```

Somente `owner` e `admin` devem conceder ou remover acessos.

5.4 Acessar o Studio com segurança

O Studio não deve ficar público. Use um túnel SSH:

```
ssh -L 54353:127.0.0.1:54353 opc@64.181.178.125
```

Depois acesse <http://127.0.0.1:54353> no computador local.

6. Configuração da Meta para operação real

6.1 Contas necessárias

- Meta Business Portfolio verificado.
- Aplicativo Meta do tipo Business.
- Produto Instagram Graph API/Messaging.
- Conta Instagram Profissional vinculada a uma Página do Facebook.
- Usuário administrador ou desenvolvedor do app durante testes.

6.2 Webhook

1. No painel Meta, use a URL:

<https://wal-chat.64.181.178.125.nip.io/api/public/webhooks/instagram>

2. Use como verify token o valor `META_VERIFY_TOKEN` de `.env.production`.
3. Assine os campos:

- `messages`
- `messaging_postbacks`
- `comments`
- `mentions`
- `message_reactions`

4. Gere um evento de teste e confirme HTTP `200` nos logs.
5. Solicite permissões e Advanced Access exigidos pela Meta antes do Live Mode.

6.3 Tokens

Preencha em `.env.production`:

```
META_APP_ID=...  
META_APP_SECRET=...  
META_ACCESS_TOKEN=...  
META_PUBLISH_TOKEN=...  
DEMO_MODE=false
```

Depois recrie os containers:

```
docker compose --env-file .env.production \  
-f docker-compose.production.yml up -d --force-recreate
```

O MVP aceita tokens de backend. Para múltiplos clientes reais, cada workspace deve concluir o OAuth da Meta e armazenar o token criptografado em `private.instagram_credentials`; não compartilhe um único token entre tenants.

7. Configuração do Gemini

1. Crie um projeto dedicado no Google AI Studio/Google Cloud.
2. Gere uma chave exclusiva para o Wal Chat.
3. Restrinja orçamento, alertas e uso da chave.
4. Preencha `GOOGLE_GENERATIVE_AI_API_KEY` em `.env.production`.
5. Recrie `app`, `webhooks` e `scheduler`.
6. No Playground do agente, teste respostas em PT-BR sem dados sensíveis.

O modelo configurado é `gemini-2.5-flash`.

8. Regras obrigatórias antes de disparos

- Toda mensagem automática recebe o rodapé `Responda PARAR`.
- Mensagens automáticas fora da janela de 24 horas são bloqueadas.
- `HUMAN_AGENT` é reservado a atendimento humano e limitado a sete dias.
- Um mesmo gatilho respeita cooldown de 24 horas por contato.
- Cada comentário recebe no máximo uma Private Reply.
- Opt-out e blocklist são verificados antes do envio.
- Campanhas revalidam elegibilidade no momento de cada disparo.
- A taxa de reengajamento deve permanecer entre 30 e 45 mensagens/minuto.

9. Validação funcional

9.1 Testes automatizados

```
npm test  
npm run lint  
npx tsc --noEmit  
npm run build
```

9.2 Rotas e saúde

```
SMOKE_APP_URL=http://127.0.0.1:3000 npm run validate:routes
```

9.3 Pipeline completo

Dentro do container `app`:

```
node scripts/smoke.mjs
```

Esse teste valida Auth, RLS, webhook GET, assinatura HMAC, rejeição de assinatura inválida, BullMQ, worker, gatilho e scheduler.

9.4 Checklist manual por módulo

- **Dashboard:** cards, gráfico, movimentos e atalhos.
- **Inbox:** abas, conversa, janela Meta e sugestão de IA.
- **Contatos:** busca, tags e exportação CSV.
- **Gatilhos:** ativar/desativar e testar palavra-chave.
- **Sequências:** blocos, delay e ordenação.
- **Agentes:** persona, modo copilot/autônomo e playground.
- **Reengajamento:** filtro, preview, elegibilidade e limite de taxa.
- **Calendário:** mês/semana e drag-and-drop.
- **Publicar:** feed, reel, story e carrossel.
- **Auto-like:** todos, sentimento positivo e palavra-gatilho.
- **Insights:** crescimento, heatmap, top posts e análise IA.
- **Compliance:** páginas legais e exclusão de dados.

10. Operação diária

Estado dos serviços

```
cd /opt/wal-chat/app
docker compose --env-file .env.production \
  -f docker-compose.production.yml ps
```

Logs

```
docker compose --env-file .env.production \
  -f docker-compose.production.yml logs --tail=200 app webhooks scheduler
```

Reinício seguro

```
docker compose --env-file .env.production \
  -f docker-compose.production.yml restart app webhooks scheduler
```

Atualização

1. Fazer backup do banco.
2. Enviar o código validado para `/opt/wal-chat/app`.
3. Copiar migrations novas para a instância Supabase.

4. Executar `supabase migration up` na instância isolada.
5. Recriar a imagem da aplicação.
6. Rodar validação de rotas e smoke test.
7. Confirmar logs sem erros.

11. Backup e recuperação

Fazer backup diário do Postgres e dos volumes de Storage. Para uma cópia lógica:

```
cd /opt/wal-chat/supabase-instance
npx --yes supabase@2.109.1 db dump --local \
  --file /opt/wal-chat/backups/wal-chat-$(date +%F).sql
```

Manter ao menos sete backups diários e quatro semanais fora do servidor principal. Testar restauração mensalmente.

12. Limites conhecidos do MVP

- Telas analíticas e editoriais apresentam dados de demonstração até a coleta real de Insights estar conectada à conta Meta.
- OAuth multi-tenant da Meta depende do App ID, segredo, URLs de callback e aprovação da Meta.
- Publicação real e geração de imagens exigem tokens Meta e chave Gemini válidos.
- SMTP deve ser configurado antes de exigir confirmação de e-mail ou recuperação de senha.
- A entrada em Live Mode depende de verificação empresarial, análise de permissões e políticas da Meta.

Esses itens não devem ser simulados como produção concluída. Até a entrega das credenciais externas, mantenha `DEMO_MODE=true`.